UG3 / UG4 GROUP PROJECT · P05 · MODULES 1–4

Text-to-SQL Data Analyst with Execution-Guided Self-Correction

An agent that turns business questions into SQL, runs it against a real SQLite database, repairs its own errors, and explains the result.

Level Advanced | **System** Single agent (with a repair loop) | **RAG** Yes | **Fine-tuning** Yes

Deliverable 1 notebook + 10-min talk + 5-min Q&A

SECTION 01

Problem

Non-technical staff want answers from databases without writing SQL. Build an analyst agent that writes a query from the question and schema, executes it, reads any error, fixes the query, and explains the answer. Then test whether fine-tuning a small model beats few-shot prompting.

Decision	Choice for this project	Why
Single or multi-agent	Single agent (with a repair loop)	One reasoning chain with an execute tool; the interesting part is self-correction, not coordination.
RAG	Yes	Required, as example retrieval: retrieve the most similar (question, schema, SQL) training examples as few-shot demonstrations.
Fine-tuning	Yes	Required. LoRA-fine-tune a 0.5B model on 5–10k training pairs and compare with few-shot prompting of the 1.5B model.

SECTION 02

Dataset (Hugging Face, public)

Dataset ID	Config / split	Columns	Role
`gretelai/synthetic_text_to_sql`	default / train, test	sql_prompt, sql_context (CREATE + INSERT statements), sql, sql_explanation, domain, sql_complexity	~100k examples across 100 domains; each sql_context builds a runnable SQLite database

```
from datasets import load_dataset
ds = load_dataset("gretelai/synthetic_text_to_sql")
# For each example: run sql_context in an in-memory sqlite3 DB, then compare your query's result with the gold `sql` result
```

> **Rule:** the notebook must download the data with `load_dataset(...)` at the top, so anyone can re-run it. If you use a subset, select it with a fixed seed and state its size. Your RAG index must be built from this downloaded data, not from text pasted into the notebook.

SECTION 03

Model

Qwen/Qwen2.5-Coder-1.5B-Instruct (agent); Qwen/Qwen2.5-0.5B-Instruct or Coder-0.5B for LoRA fine-tuning. Embeddings for example retrieval: all-MiniLM-L6-v2.

> **Compute:** everything must run on a free Colab T4 GPU or a laptop CPU. Use Qwen models of 4B parameters or fewer; no paid APIs are required. Any similar open model (e.g. Llama-3.2-1B/3B-Instruct, Phi-3.5-mini, Gemma-2-2B) is acceptable if you justify the choice.

SECTION 04

System Design

```
question + schema → retrieve similar solved examples (RAG)
                  → generate SQL → execute_sql tool (sqlite, read-only)
                                 | error or empty?
                                 └→ repair with the error message (≤ 3 rounds) → explain result
```

SECTION 05

What to Build — Step by Step

1. Build an execution harness: create the DB from sql_context, run gold and predicted SQL, compare result sets.
2. Baseline: zero-shot SQL generation; then retrieved few-shot.
3. Implement execute_sql as a tool (read-only, timeout) and the repair loop in LangGraph.
4. LoRA fine-tuning (Module 1 lab) on a training subset; evaluate on the same test subset.
5. Break results down by sql_complexity and domain.

Stretch goal (for top marks): Add a chart tool that plots numeric results.

SECTION 06

Evaluation

Metrics

- Execution accuracy (result sets match)
- Exact-match accuracy (secondary)
- Repair success rate and average repair rounds

- Accuracy by sql_complexity

Baselines

Zero-shot vs few-shot RAG vs few-shot + repair loop vs LoRA model (+ repair loop).

Ablations

Repair rounds {0, 1, 3}; examples k {0, 3, 5}; LoRA training size {1k, 5k}.

> **Minimum standard:** at least one baseline, your full system, and one ablation, all on the same fixed test subset, reported in one results table, followed by an error analysis of at least 10 failure cases.

SECTION 07

Labs to Reuse

- Module 1 · Fine_Tuning_Lab (LoRA / QLoRA with peft)
- Module 2 · Structured_Generation_Lab (CoT, self-consistency, PoT, ReAct, reflection, outlines)
- Module 3 · RAG_Pipeline_Lab (chunking, FAISS, BM25, hybrid RRF, cross-encoder re-ranking, Recall@k / MRR)
- Module 4 · Multi_Agent_System_End_to_End (tools, ReAct graph, memory, supervisor, human-in-the-loop, MCP)

Start from the lab code; you are expected to adapt it, not rewrite it from scratch. Cite which lab each part of your notebook builds on.

> **Read this brief together with the Group Project Guidelines & Judging document**, which sets the notebook structure, the presentation format (10 minutes + 5 minutes Q&A) and how projects are marked.
